

Chap 8. Array_String

Contents

01 블랙박스 테스트 개요

02

01

Arrays

- If you need 4 identical data variables



```
1 int main(){  
2     int a1=10, a2=20, a3=30, a4=40;  
3 }
```



```
1 int main(){  
2     int myNumbers[] = {25, 50, 75, 100};  
3     printf("%d", myNumbers[0]);  
4 }
```

Arrays

- Arrays are used to store multiple values in a single variable, instead of declaring separate variables for each value.
- To create an array, define the data type (like int) and specify the name of the array followed by square brackets [].
- To insert values to it, use a comma-separated list inside curly braces, and make sure all values are of the same data type:

Arrays



```
1 int main(){
2     int myNumbers[] = {25, 50, 75, 100};
3     myNumbers[0] = 33;
4
5     printf("%d", myNumbers[0]);
6     return 0
7 }
```

Arrays

■ Set Array Size

- to create arrays, is to specify the size of the array, and add elements later
- not able to change the size of the array after creation.

```
1 int main(){
2     // Declare an array of four integers:
3     int myNumbers[4];
4
5     // Add elements
6     myNumbers[0] = 25;
7     myNumbers[1] = 50;
8     myNumbers[2] = 75;
9     myNumbers[3] = 100;
10    return 0;
11 }
```

Arrays

■ Avoid Mixing Data Types

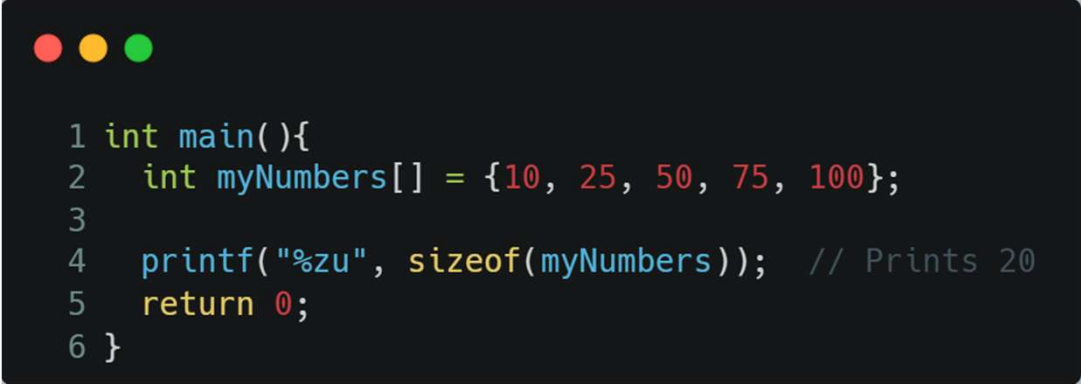
- all elements in an array **must be of the same [data type](#)**.
- you cannot mix different types of values, like **integers** and **floating point** numbers, in the same array:

```
int myArray[] = {25, 50, 75, 3.15, 5.99};
```


Arrays

■ Get Array Size

- To get the size of an array, you can use the sizeof operator:



```
1 int main(){
2     int myNumbers[] = {10, 25, 50, 75, 100};
3
4     printf("%zu", sizeof(myNumbers)); // Prints 20
5     return 0;
6 }
```

- Why did the result show 20 instead of 5, when the array contains 5 elements?
 - the sizeof operator returns the size of a type in bytes.
 - 4 x 5 (4 bytes x 5 elements) = 20 bytes.

Arrays

■ Get the Number of Elements

- If you want to find out how many elements an array has, you can use this formula, which divides the total size of the array by the size of one element

```
1 int main(){
2     int myNumbers[] = {10, 25, 50, 75, 100};
3     int length = sizeof(myNumbers) / sizeof(myNumbers[0]);
4
5     printf("%d", length); // Prints 5
6     return 0;
7 }
```

Arrays

■ Get the Number of Elements

- The sizeof formula works for arrays of any type and any size:

```
1 int main(){
2     double myValues[] = {1.1, 2.2, 3.3};
3     int length = sizeof(myValues) / sizeof(myValues[0]);
4     // length = sizeof(myValues) / sizeof(double);
5     printf("%d", length); // Prints 3
6     return 0;
7 }
```

Arrays Loops

■ Loop Through an Array

- You can use a for loop to go through the elements of an array by writing the size of the array in the loop condition (in this example the array has 4 elements, so we use $i < 4$). However, this is not ideal, since it will only work for arrays of a specified size:

```
1 int main(){
2     int myNumbers[] = {25, 50, 75, 100};
3     int i;
4
5     for (i = 0; i < 4; i++) {
6         printf("%d\n", myNumbers[i]);
7     }
8     return 0;
9 }
```

Arrays Loops

■ Making Better Loops

- you learned how to calculate the number of elements in an array using the sizeof formula
- Now we can use that to write loops that work for arrays of any size. This is more flexible and sustainable:

```
1 #include <stdio.h>
2
3 int main() {
4     int myNumbers[] = {25, 50, 75, 100};
5     int length = sizeof(myNumbers) / sizeof(myNumbers[0]);
6     int i;
7
8     for (i = 0; i < length; i++) {
9         printf("%d\n", myNumbers[i]);
10    }
11
12    return 0;
13 }
```

Arrays Examples(1)

- let's create a program that calculates the average of different ages:

```
1  #include <stdio.h>
2  int main(){
3      // An array storing different ages
4      int ages[] = {20, 22, 18, 35, 48, 26, 87, 70};
5
6      float avg, sum = 0;
7      int i;
8
9      // Get the length of the array
10     int length = sizeof(ages) / sizeof(ages[0]);
11
12     // Loop through the elements of the array
13     for (i = 0; i < length; i++) {
14         sum += ages[i];
15     }
16
17     // Calculate the average by dividing the sum by the length
18     avg = sum / length;
19
20     // Print the average
21     printf("The average age is: %.2f", avg);
22
23     return 0;
24 }
```

Arrays Examples(2)

- finds the lowest age among different ages:

```
// An array storing different ages
int ages[] = {20, 22, 18, 35, 48, 26, 87, 70};

int i;

// Get the length of the array
int length = sizeof(ages) / sizeof(ages[0]);

// Create a variable and assign the first array element of ages to it
int lowestAge = ages[0];

// Loop through the elements of the ages array to find the lowest age
for (i = 0; i < length; i++) {
    if (lowestAge > ages[i]) {
        lowestAge = ages[i];
    }
}
```

Arrays Examples(3)

- We skip negative values, but stop the loop completely if we find a zero:

```
int myNumbers[] = {3, -1, 7, 0, 9};
int length = sizeof(myNumbers) / sizeof(myNumbers[0]);
int i;

for (i = 0; i < length; i++) {
    if (myNumbers[i] < 0) {
        continue; // skip negative numbers
    }
    if (myNumbers[i] == 0) {
        break; // stop loop when zero is found
    }
    printf("%d\n", myNumbers[i]);
}
```


Multidimensional Arrays

■ Multidimensional Arrays

- if you want to store data as a tabular form, like a table with rows and columns, you need to get familiar with **multidimensional arrays**.
- A multidimensional array is basically an array of arrays

■ Two-Dimensional Arrays

- A 2D array is also known as a matrix (a table of rows and columns).
- To create a 2D array of integers, take a look at the following example:

```
int matrix[2][3] = { {1, 4, 2}, {3, 6, 8} };
```

	COLUMN 0	COLUMN 1	COLUMN 2
ROW 0	1	4	2
ROW 1	3	6	8

Multidimensional Arrays

■ Access the Elements of a 2D Array

- To access an element of a two-dimensional array, you must specify the index number of both the row and column.
- This statement accesses the value of the element in the first row (0) and third column (2) of the matrix array.

```
int matrix[2][3] = { {1, 4, 2}, {3, 6, 8} };  
  
printf("%d", matrix[0][2]); // Outputs 2
```

```
int matrix[2][3] = { {1, 4, 2}, {3, 6, 8} };  
matrix[0][0] = 9;  
  
printf("%d", matrix[0][0]); // Now outputs 9 instead of 1
```

Multidimensional Arrays

■ Loop Through a 2D Array

- To loop through a multi-dimensional array, you need one loop for each of the array's dimensions.
- The following example outputs all elements in the matrix array:

```
int matrix[2][3] = { {1, 4, 2}, {3, 6, 8} };

int i, j;
for (i = 0; i < 2; i++) {
    for (j = 0; j < 3; j++) {
        printf("%d\n", matrix[i][j]);
    }
}
```

Multidimensional Arrays

■ When to Use Multidimensional Arrays

- Multidimensional arrays are useful when your data is arranged in rows and columns, like a table, grid, or matrix.
- Each extra dimension adds another level of structure:
 - 2D arrays (like `int scores[3][4]`) are great for storing things like *scores*, *game boards*, or *spreadsheets*
 - 3D arrays (like `int cube[2][3][4]`) can represent more complex structures like a set of *tables* or levels in a *game*

code challenge

- Complete C Code
 - next data structures scores
 - { 85, 72, 92, 48, 70, 56 }
 - print Average of scores
 - find lowest score of scores
 - find max score of scores

Thank You !